Tecnologie e architetture per la gestione dei dati (ex Basi di dati II)

P. Atzeni (con materiale di S. Ceri)

Tecnologia delle BD: perché studiarla?

- I DBMS offrono i loro servizi in modo "trasparente":
  - per questo abbiamo potuto finora ignorare molti aspetti realizzativi
    - Struttura logica vs struttura fisica
    - Interrogazioni ad alto livello (SQL)
    - Transazioni
- Abbiamo considerato il DBMS come una "scatola nera"

Tecnologia delle BD: perché studiarla? (2)

- Abbiamo considerato il DBMS come una "scatola nera"
  - Come un'automobile senza conoscere i dettagli del funzionamento
- Perché approfondire e aprire la scatola nera?
  - ingegneri informatici debbono sapere che cosa utilizzano e saper realizzare DBMS … (anche se in Italia pochi o nessuno lo fanno)
  - capire come funziona può favorire un utilizzo efficace e lo sviluppo di applicazioni efficienti
  - ci sono servizi che vengono offerti separatamente

DataBase Management System — DBMS

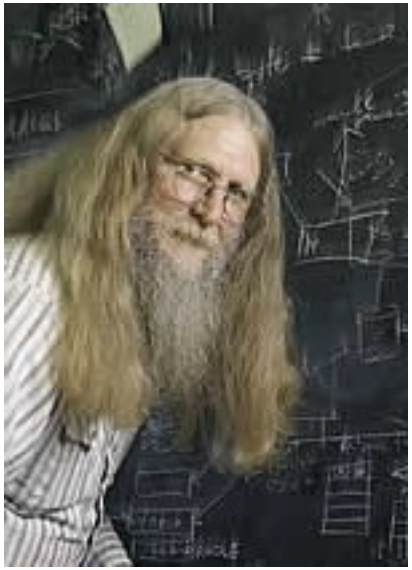
Sistema (prodotto software) in grado di gestire collezioni di dati che siano (anche):

- **grandi** (di dimensioni (molto) maggiori della memoria centrale dei sistemi di calcolo utilizzati)
- **persistenti** (con un periodo di vita indipendente dalle singole esecuzioni dei programmi che le utilizzano)
- **condivise** (utilizzate da applicazioni diverse)

garantendo **affidabilità** (resistenza a malfunzionamenti hardware e software) e privatezza (con una disciplina e un controllo degli accessi). Come ogni prodotto informatico, un DBMS deve essere **efficiente** (utilizzando al meglio le risorse di spazio e tempo del sistema) ed efficace (rendendo produttive le attività dei suoi utilizzatori).

Efficienza

- *There are three problems in database systems:*
  - *Performance*
  - *Performance*
  - *Performance*



Bruce Lindsay, IBM

Le basi di dati sono grandi e persistenti

- La persistenza richiede una gestione in memoria secondaria
- La grandezza richiede che tale gestione sia sofisticata (non possiamo caricare tutto in memoria principale e poi riscaricare)

Le basi di dati vengono interrogate …

- Gli utenti vedono il modello logico (relazionale)
- I dati sono in memoria secondaria
- Le strutture logiche non sarebbero efficienti in memoria secondaria:
  - servono strutture fisiche opportune
- La memoria secondaria è molto più lenta della memoria principale:
  - serve un'interazione fra memoria principale e secondaria che limiti il più possibile gli accessi alla secondaria

Esempio

- Interrogazione con un join:
  - due cursori nidificati oppure un solo cursore con una interrogazione che fa il join?

Lists(<u>ListName</u>, Owner, CreationDate, …)

ListParticipants(<u>ListName</u>, <u>Email</u>, RegistrationDate, …)

Per ogni lista, l'elenco dei partecipanti. Che cosa non va bene?

Lists(ListName, Owner, CreationDate, …)
ListParticipants(ListName, Email, RegistrationDate, …)

```
Statement statement1 = connection.createStatement();
Statement statement2 = connection.createStatement();
ResultSet resultSet1 = statement1.executeQuery("select * from Lists");
while(resultSet1.next()){
        String listname = resultSet1.getString("ListName");
        ResultSet resultSet2 =
                statement2.executeQuery("select * " +
                                "from ListParticipants " +
                                "where ListName= '"+listname+"'");
        while(resultSet2.next()){
                // get the participants
                …
        }
}
```

Facciamo fare il join al DBMS!

Lists(ListName, Owner, CreationDate, …)

ListParticipants(ListName, Email, RegistrationDate, …)

```
Statement statement0 = connection.createStatement();
ResultSet resultSet0 =
        statement0.executeQuery(
                        "select * " +
                        "from Lists L join ListParticipants LP " +
                        "   on L.ListName=LP.ListName");
        while(resultSet0.next()){
        …
        }
}
```

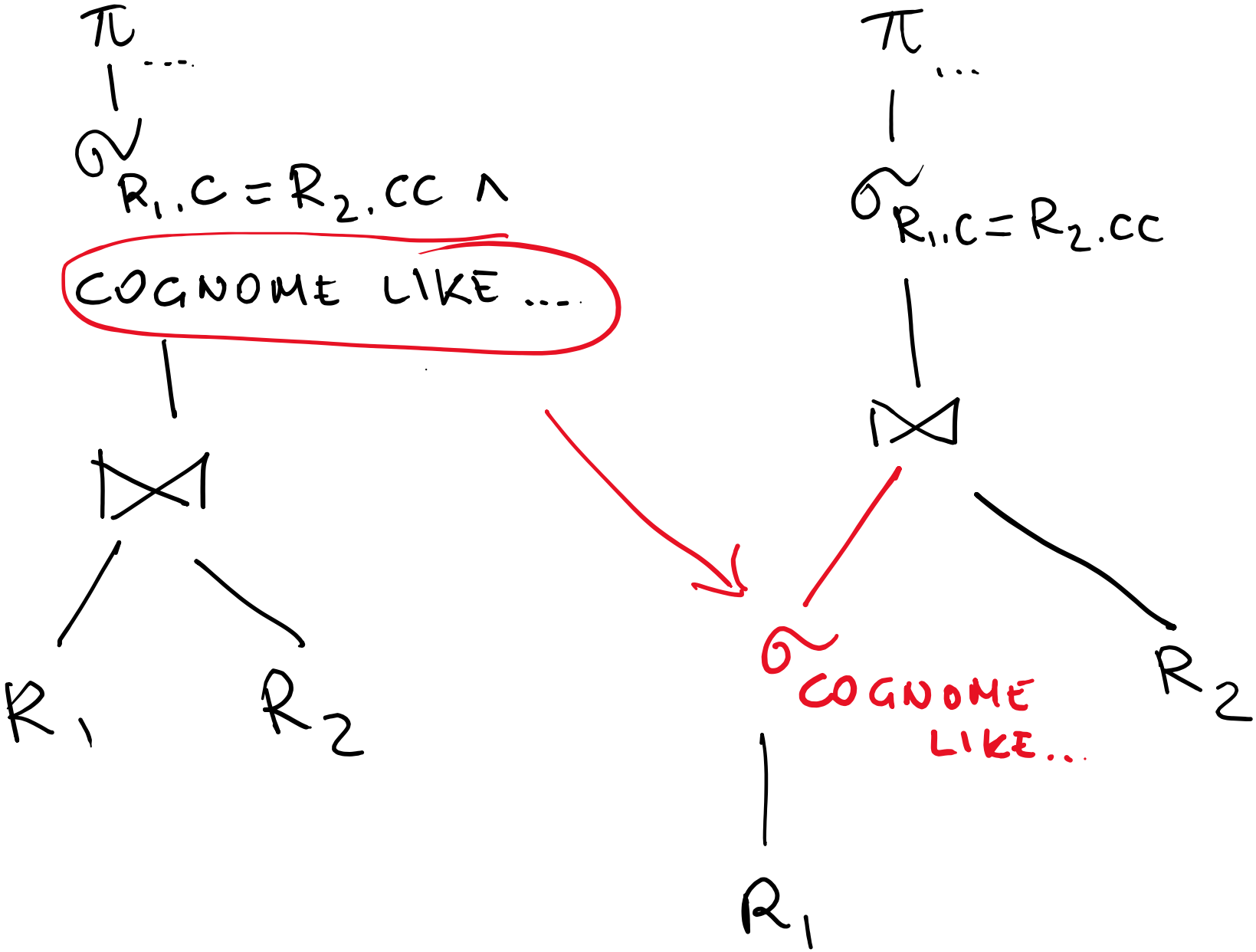
Altro esempio

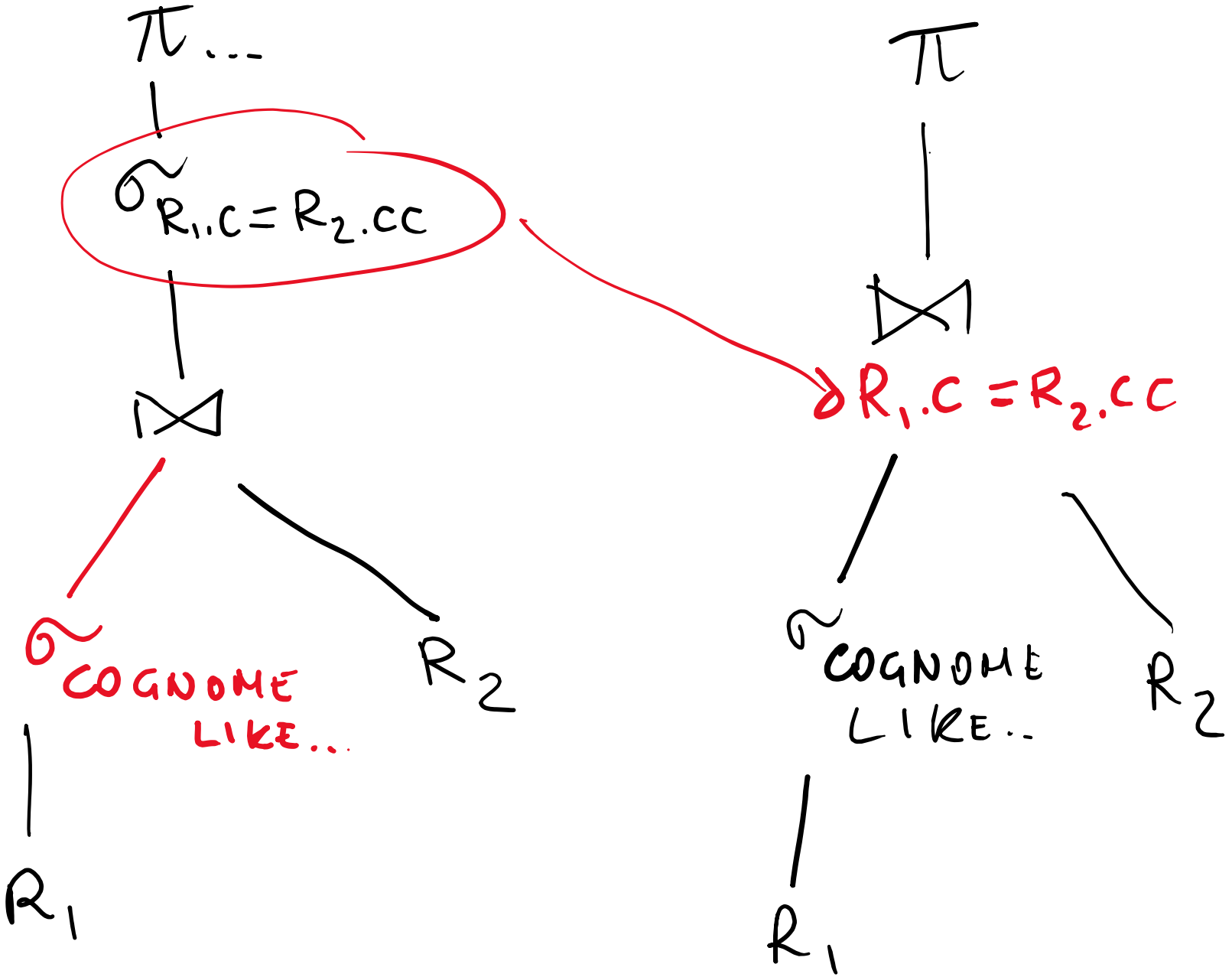
R1(<u>Matricola</u>,Cognome,Comune)

R2(<u>CodiceComune</u>,Comune,Provincia)

```
SELECT R1.Matricola, R1.Cognome,
            R2.CodiceComune, R2.Comune
FROM R1, R2
WHERE R1.Comune=R2.CodiceComune
    AND Cognome LIKE 'Pallino%'
```

- Le relazioni hanno una "struttura fisica"
- I dati sono organizzati in blocchi
- Le interrogazioni vengono implementate con opportuni algoritmi

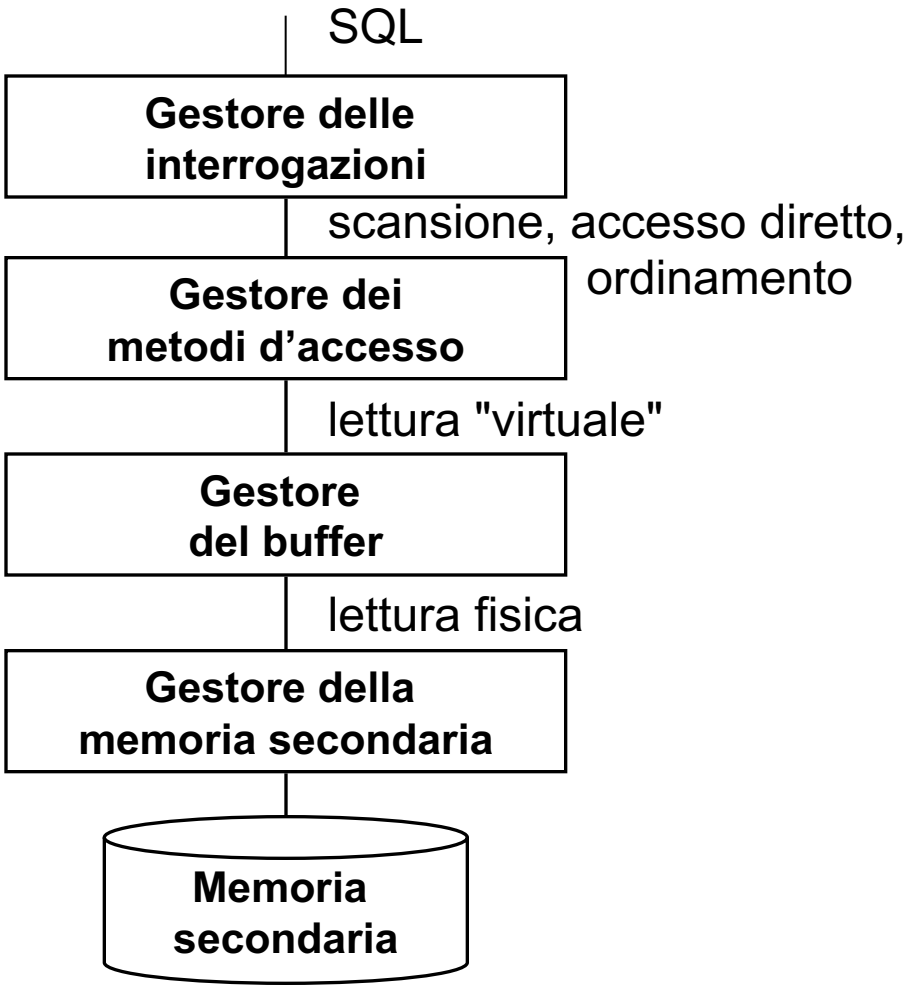

π ...
σ R1.C = R2.CC ∧
COGNOME LIKE ...
⋈
R1
R2
π ...
σ R1.C = R2.CC
⋈
σ COGNOME LIKE...
R2
R1


π ...
σ R1.C = R2.CC
⋈
σ COGNOME LIKE...
R2
R1
π
⋈
σ R1.C = R2.CC
σ COGNOME LIKE..
R2
R1

Altri esempi

- Come cercate un libro in una libreria o in una biblioteca?
- Come ordinereste un pacchetto di centocinquanta documenti (ad esempio compiti d'esame)?

Gestore degli accessi e delle interrogazioni



Le basi di dati sono affidabili

- Le basi di dati sono una risorsa per chi le possiede, e debbono essere conservate anche in presenza di malfunzionamenti

- Esempio:
  - un trasferimento di fondi da un conto corrente bancario ad un altro, con guasto del sistema a metà

- Le **transazioni** debbono essere
  - atomiche (o tutto o niente)
  - definitive: dopo la conclusione, non si dimenticano

Esempio

- Debbo prenotare un viaggio
  - Aereo
  - Albergo
  - Automobile a noleggio

- Confermo e mi impegno a pagare solo se riesco ad avere disponibilità per tutti e tre

Le basi di dati vengono aggiornate …

- L'**affidabilità** è impegnativa per via degli aggiornamenti frequenti e della necessità di gestire il buffer

Le basi di dati sono condivise

- Una base di dati è una risorsa **integrata**, **condivisa** fra le varie applicazioni
- conseguenze
  - Attività diverse su dati in parte condivisi:
    - meccanismi di autorizzazione
  - Attività multi-utente su dati condivisi:
    - controllo della **concorrenza**

Aggiornamenti su basi di dati condivise …

- Esempi:
  - due prelevamenti (quasi) contemporanei sullo stesso conto corrente
  - due prenotazioni (quasi) contemporanee sullo stesso posto

Due prelevamenti da un conto corrente

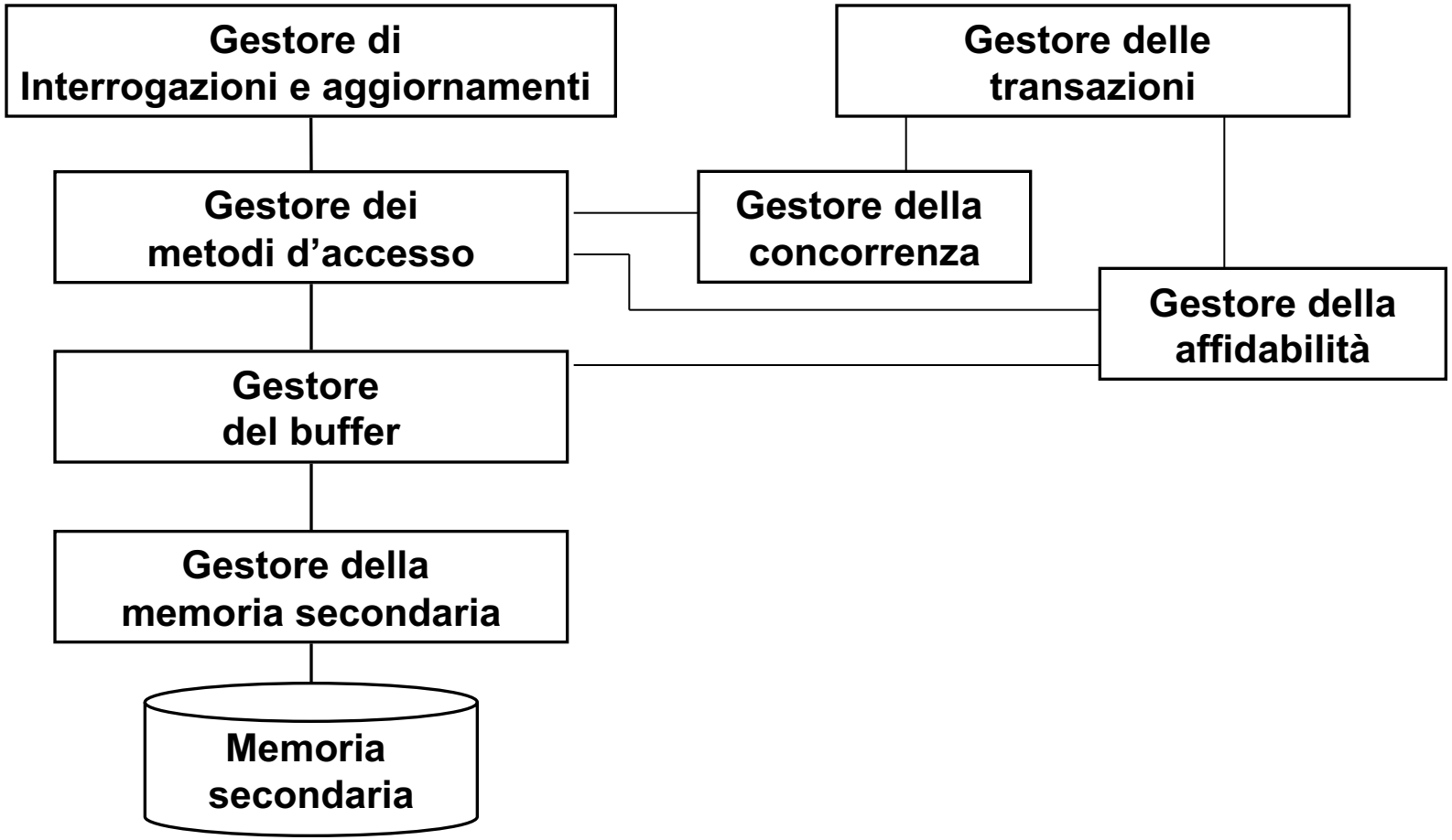
- Quanto c'è sul conto?
  - 1000
- Quanto c'è sul conto?
  - 1000
- Bene, allora prelevo 200
  - Ok, il nuovo saldo è 800
- Bene, allora prelevo 100
  - Ok, il nuovo saldo è 900

Aggiornamenti su basi di dati condivise …

- …
- Intuitivamente, le transazioni sono corrette se **seriali** (prima una e poi l'altra)
- Ma in molti sistemi reali l'efficienza sarebbe penalizzata troppo se le transazioni fossero seriali:
  - il **controllo della concorrenza** permette un ragionevole compromesso

Gestore degli accessi e delle interrogazioni

Gestore delle transazioni

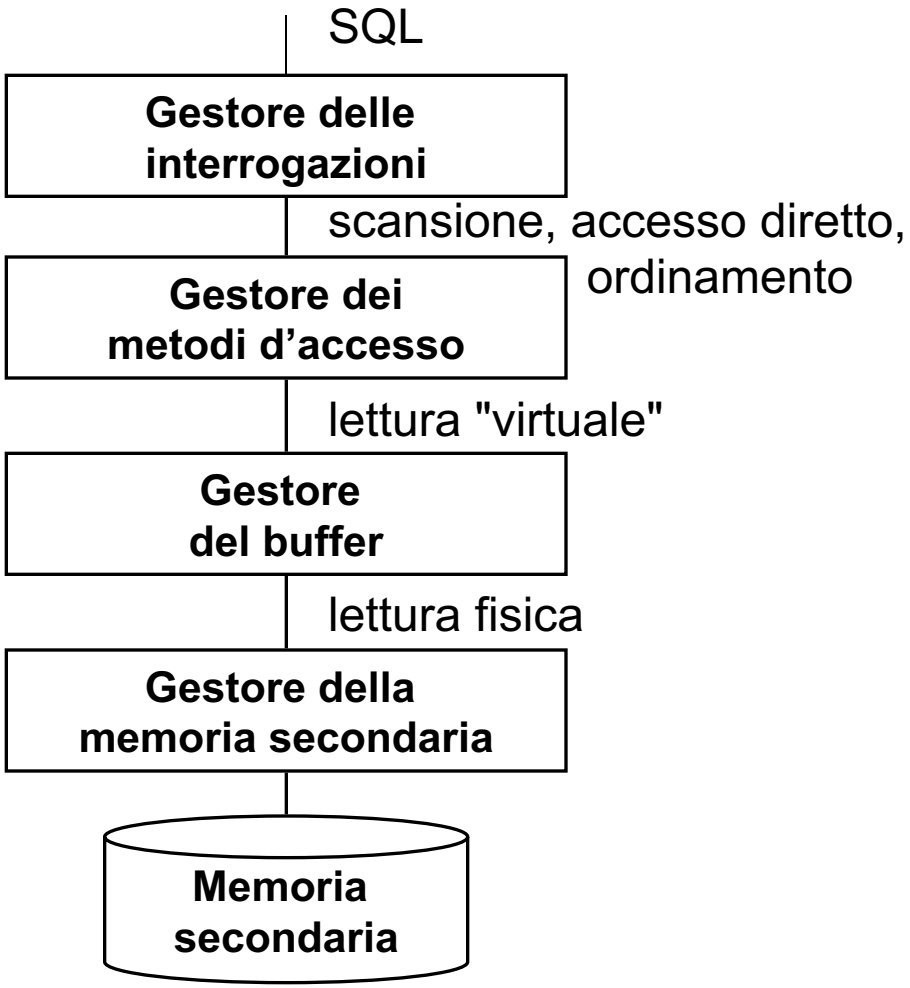


Tecnologia delle basi di dati, argomenti

- Gestione della memoria secondaria e del buffer
- Organizzazione fisica dei dati
- Gestione ("ottimizzazione") delle interrogazioni
- Controllo della affidabilità
- Controllo della concorrenza

- Architetture distribuite

Gestore degli accessi e delle interrogazioni



In concreto: SimpleDB, un DBMS didattico

Remote: riceve le richieste dal client e passa SQL a Planner

Planner: chiama Parser e determina piano e lo passa al Query

Parser: analisi sintattica

Query: riceve il piano e chiama il Record per ogni tabella

Metadata: gestisce gli schemi delle tabelle

Record: gestisce i blocchi per i record delle tabelle

Transaction: gestisce la concorrenza

Buffer: mantiene in memoria le pagine per limitare gli accessi

Log: tiene traccia delle operazioni, per l'affidabilità

File: legge e scrive le pagine su disco